

MWORKS 2025a培训（五）

从MATLAB到Syslab，资产复用全流程展示

孔德才

苏州同元软控信息技术有限公司

2025年3月20日

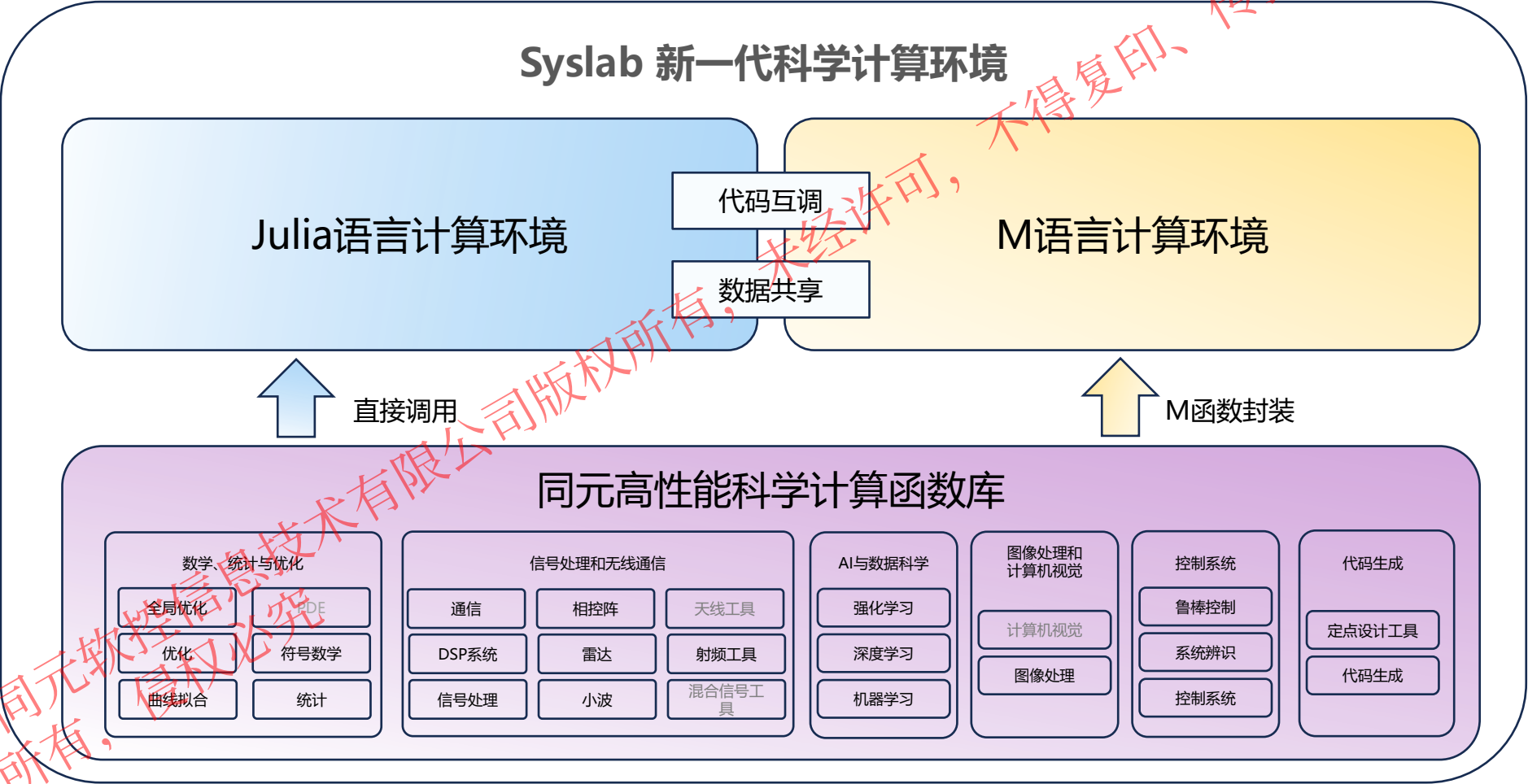


传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，
版权所有，侵权必究

Syslab与M语言计算环境

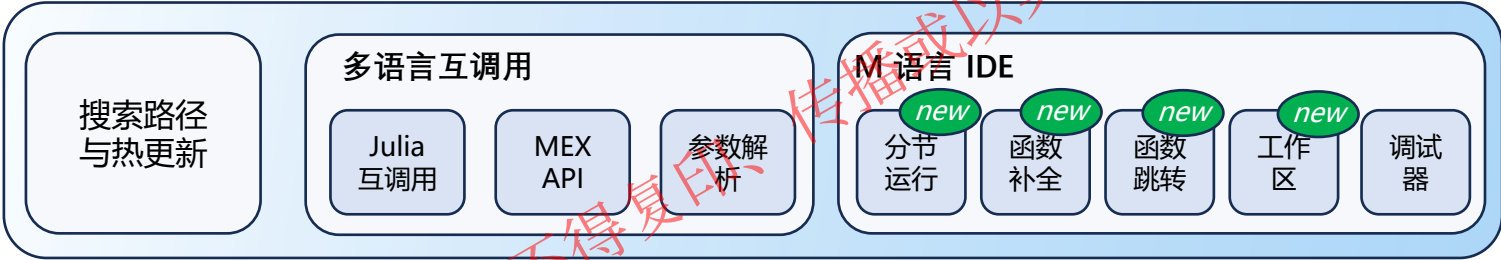
MWORKS.Syslab 是新一代科学计算环境，以Julia语言为主，同时兼容M语言，支持 Julia 与 M、Python等编程语言的相互调用



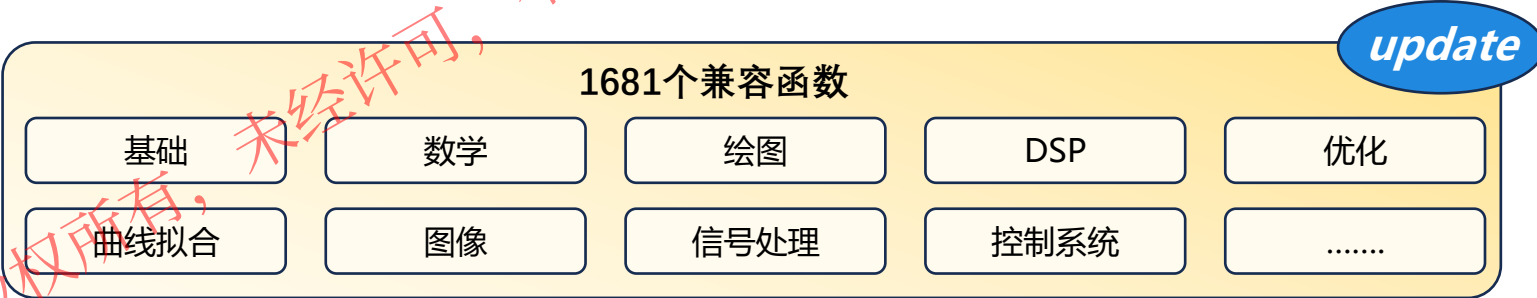
M语言计算环境：升级总览

- 内核重构，性能提升 **2~100** 倍
- **M 语言 IDE** 功能升级
- 新增 **classdef** 面向对象语法支持
- 新增 **421** 个M函数，合计 **1681** 个

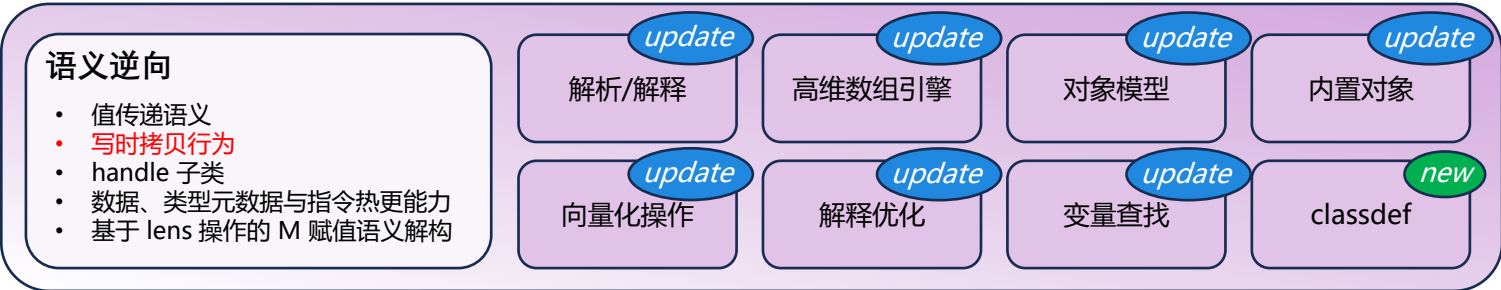
交互层



函数层



内核层



@苏州同元软控信息技术有限公司
版权所有，侵权必究

目录

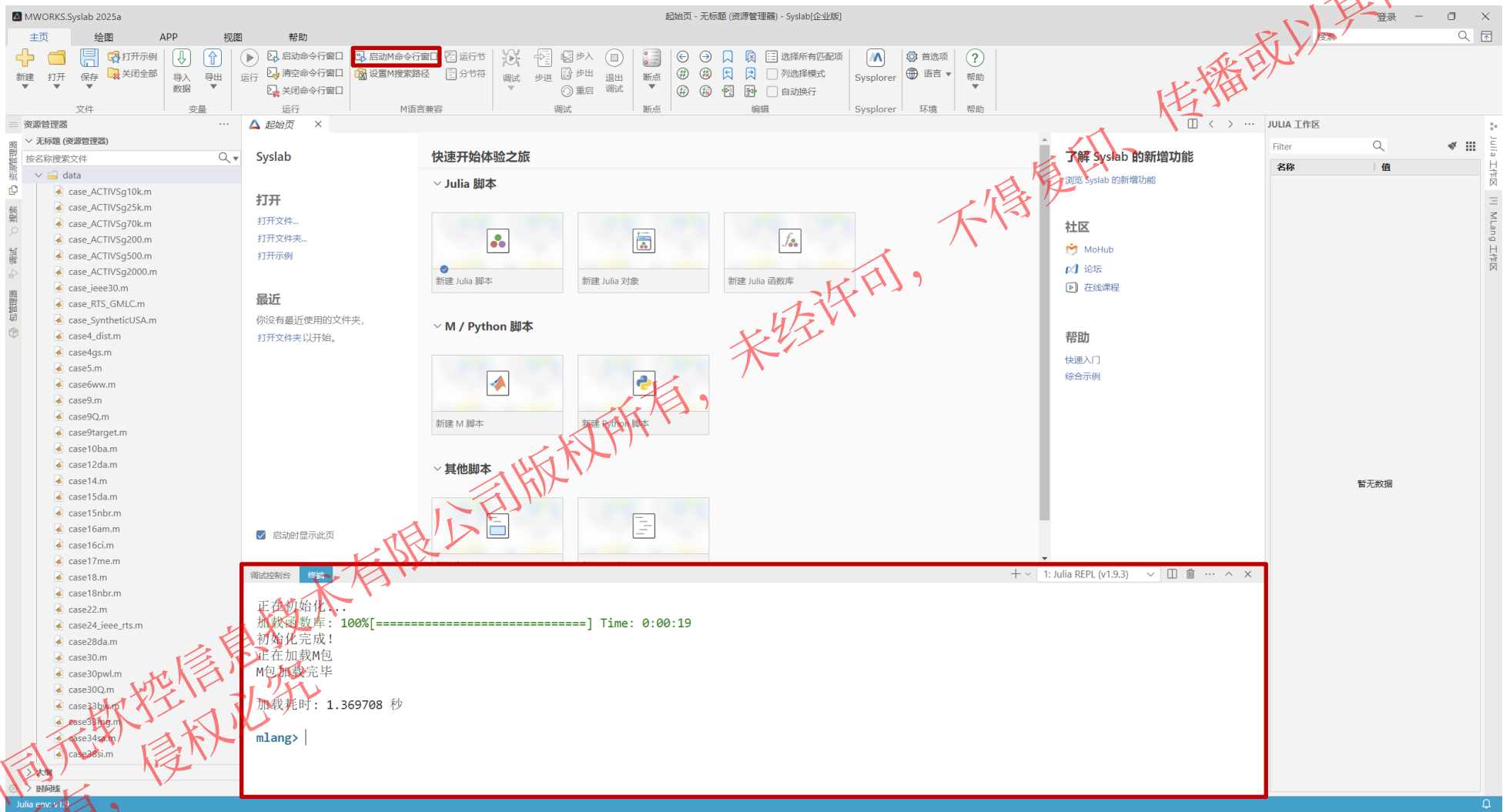
1. 基础设置

2. M脚本运行与调试

3. 高阶功能

1.1 启动

单击面板启动 M 命令行窗口按钮即可启动 M 语言计算环境命令行



1.2 设置路径

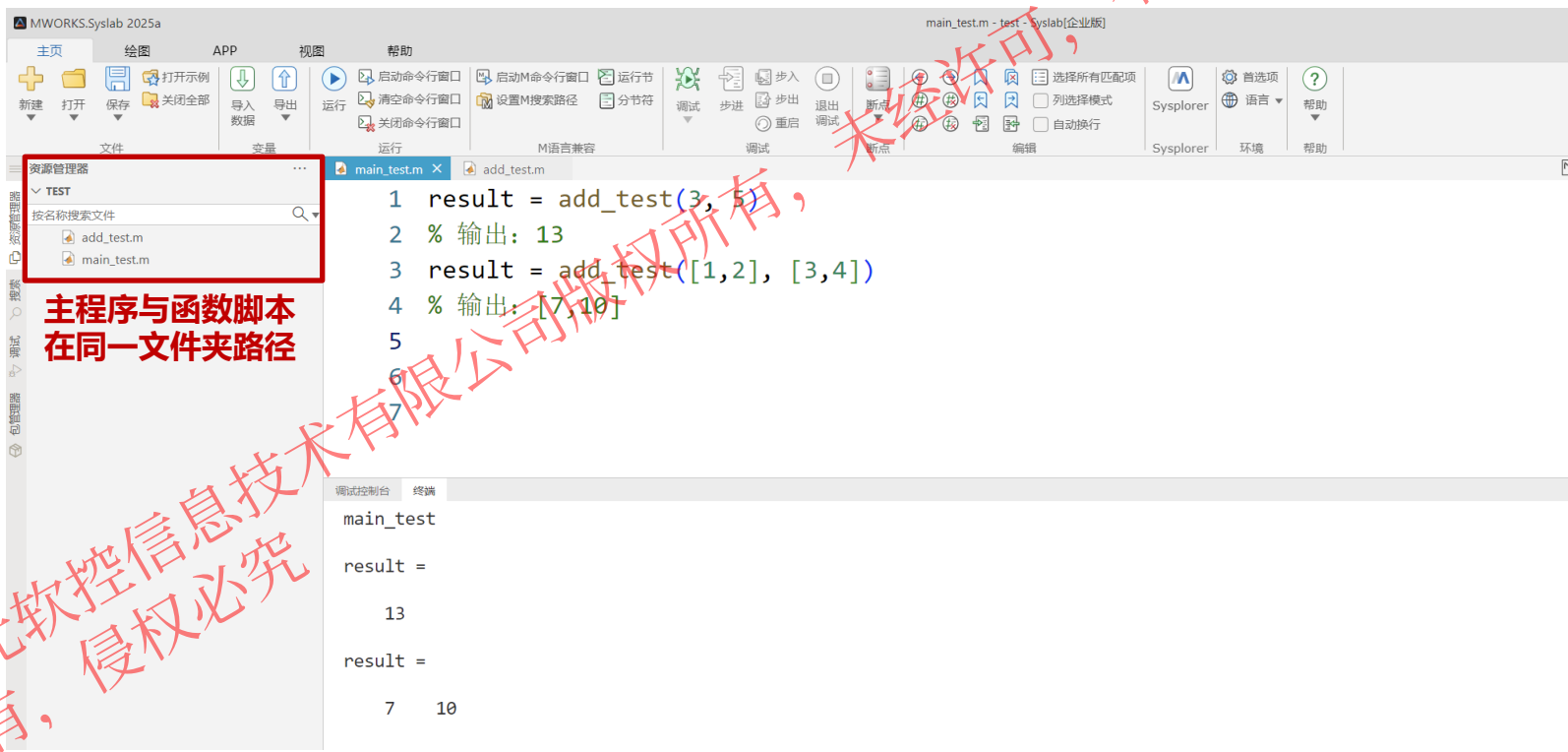
与 MATLAB 一致，资源管理器打开至主程序与函数脚本所在文件夹，即可直接实现函数调用

```
% 定义一个加法函数
function sum = add_test(a, b)
    sum = a + 2*b;
end
```

函数脚本

```
result = add_test(3, 5)
% 输出: 13
result = add_test([1,2], [3,4])
% 输出: [7,10]
```

主程序脚本



1.2 设置路径

M 语言计算环境支持**搜索路径**功能：只要 M 函数文件位于搜索路径中，可按文件名直接调用相应 M 函数。除内置函数库外，还支持**将自定义函数路径添加到搜索路径**

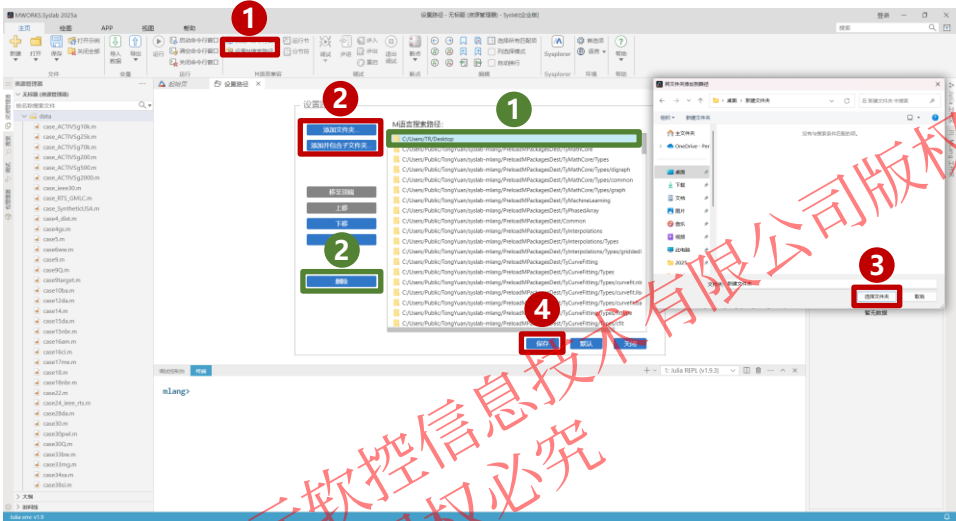
可视化操作

添加路径操作步骤：

- 1. 单击设置 M 搜索路径
- 2. 单击添加文件夹或添加并包含子文件夹
- 3. 选择目标文件夹，然后保存

删除路径操作步骤：

- 1. 选择需要删除的路径选项后单击删除，然后保存



说明：

- 添加文件夹：仅添加当前文件夹
- 添加并包含子文件夹：添加当前文件夹，并包含所有子文件夹

文本操作

添加路径操作步骤：

- 1. M 语言命令行中，使用 addpath 命令添加路径

```
mlang> addpath('D:/function')
```

删除路径操作步骤：

- 1. M 语言命令行中，使用 rmpath 命令删除路径

```
mlang> rmpath ('D:/function')
```

- 2. M 语言命令行中，使用 path 命令清空当前的搜索路径

```
mlang> path([])
```

```
mlang> addpath('D:/function')
```

添加路径

```
mlang> rmpath ('D:/function')
```

删除路径

说明：

- addpath 仅添加当前路径下的 M 文件，不添加子文件夹下的路径
- 清空路径之后重启 Syslab 命令行，搜索路径即可恢复到默认状态



目录

1. 基础设置

2. M脚本运行与调试

3. 高阶功能

2.1 MLang命令行

使用 MLang命令行时，和 M 语言语法一致，可以**创建变量、调用函数等**

demo.m

```
1 %%
2 x = linspace(0,2*pi);
3 y = sin(x);
4 plot(x,y)
5 %%
6
7 xlabel("x")
8 ylabel("sin(x)")
9 title("Plot of the Sine Function")
10
11 plot(x,y,"r--")
12
13 x = linspace(0,2*pi);
14 y = sin(x);
15 plot(x,y)
16
```

调试控制台 终端

1: Julia REPL (v1.9.3)

```
mlang> a=1
a =

 1

mlang> b=cos(pi)
b =

 -1

mlang> ones(2)
ans =
```

```
mlang> a=1
a =

 1

mlang> b=cos(pi)
b =

 -1

mlang> ones(2)
ans =

 1 1
 1 1

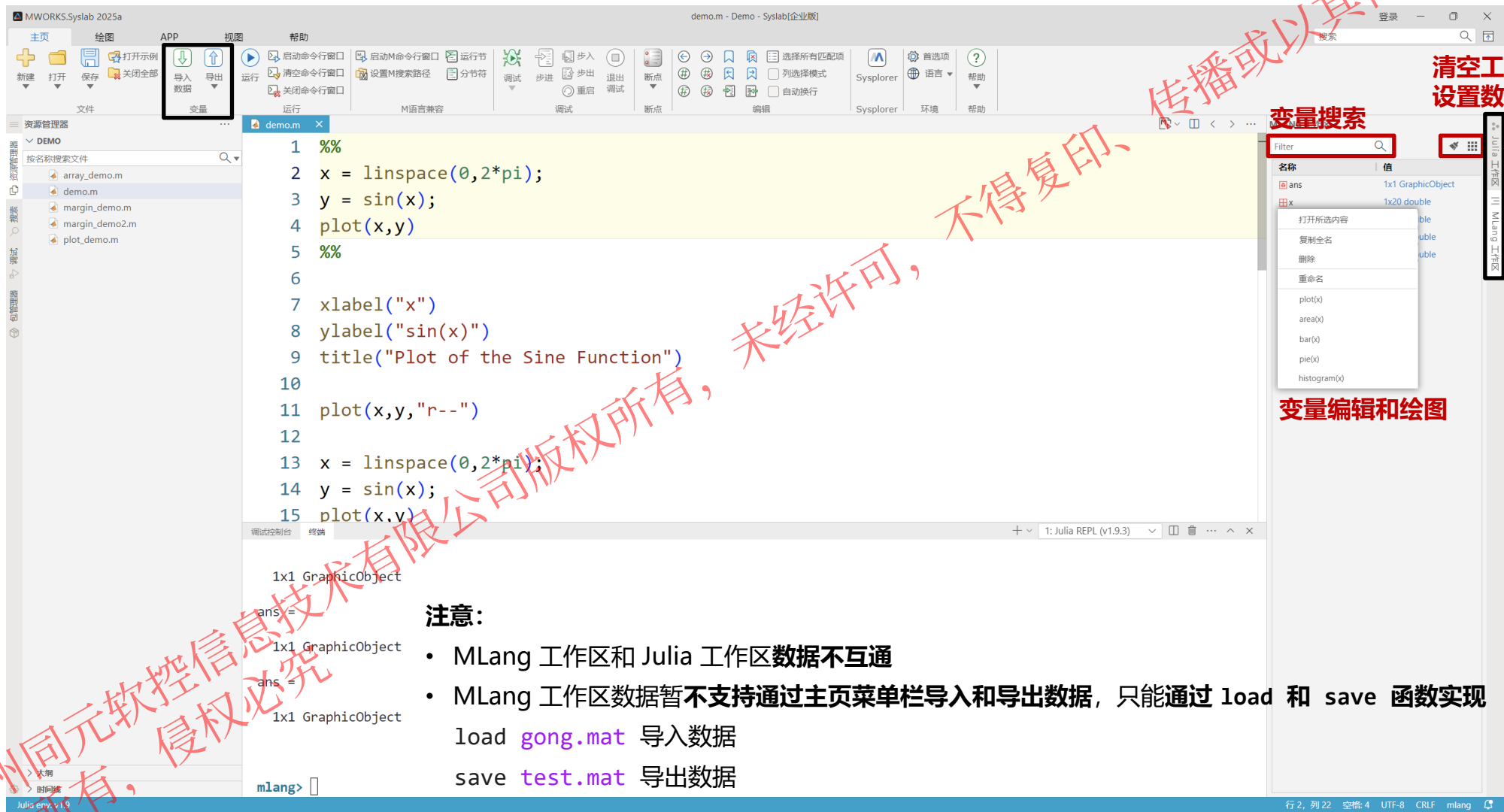
mlang> a = [1 3 5; 2 4 6; 7 8 10]
a =

 1 3 5
 2 4 6
 7 8 10
```



2.2 MLang工作区

支持对命令行窗口中变量、类型、函数等元素进行集中**显示与编辑**，支持**双击变量**打开变量编辑器



注意:

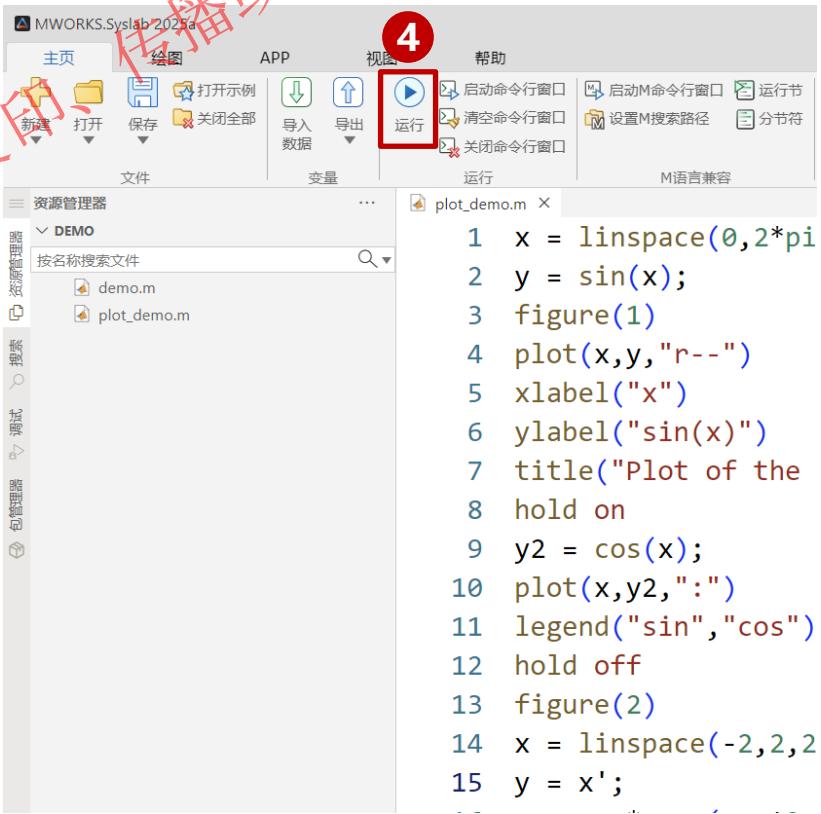
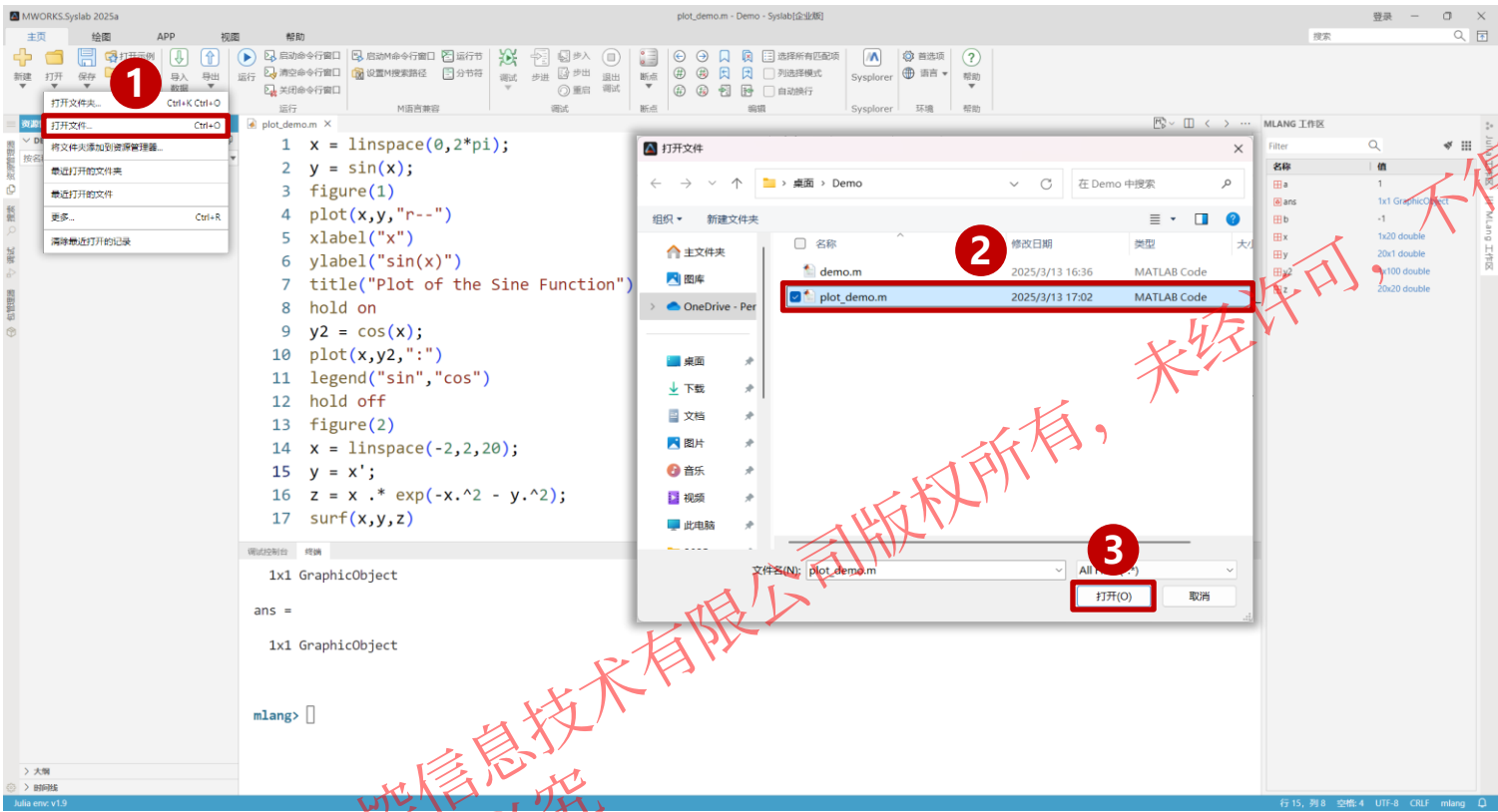
- MLang 工作区和 Julia 工作区数据不互通
 - MLang 工作区数据暂不支持通过主页菜单栏导入和导出数据，只能通过 load 和 save 函数实现
- load gong.mat 导入数据
- save test.mat 导出数据

2.3 M脚本运行

操作步骤:

1. 单击 Ribbon 菜单打开下拉按钮，选择打开文件，选择目标脚本文件

2. 单击**运行**按钮，执行该脚本

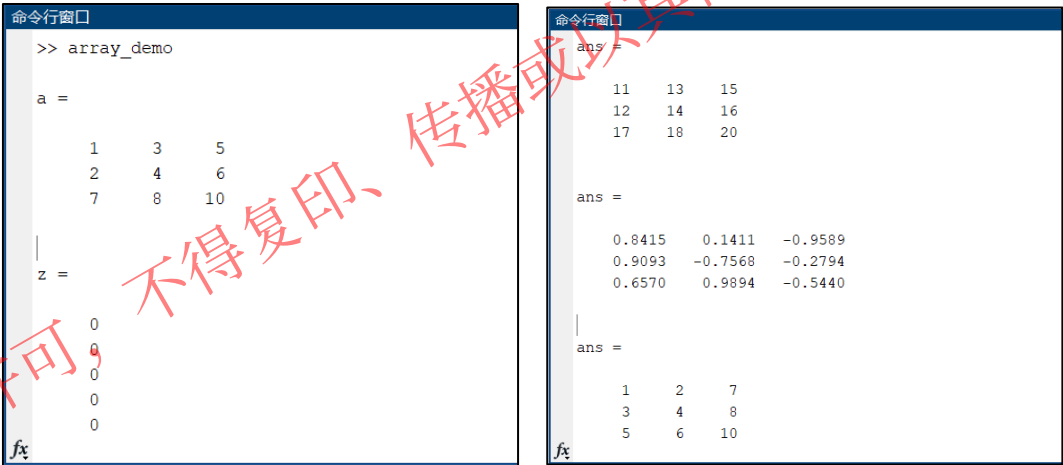


2.3 M脚本运行

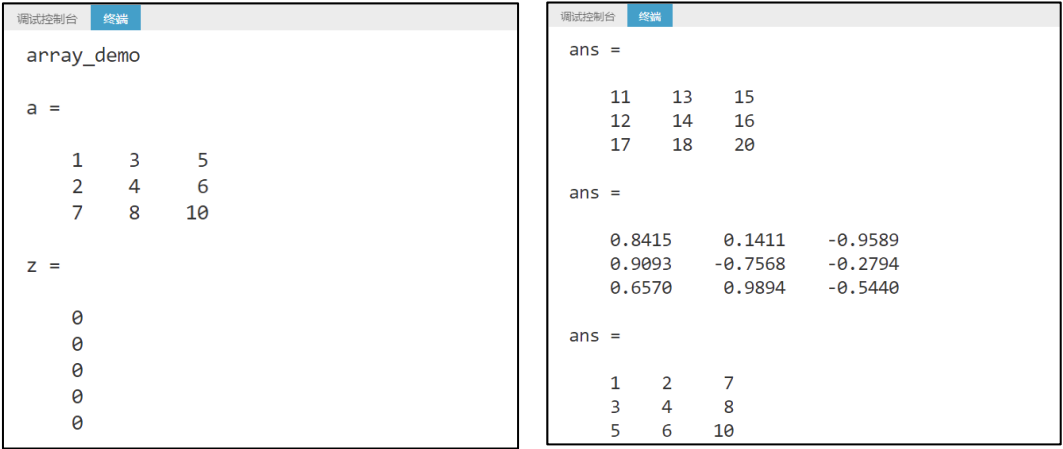
示例1：矩阵和数组

```
% 数组创建
a = [1 3 5; 2 4 6; 7 8 10]
z = zeros(5,1)
% 矩阵和数组运算
a + 10
sin(a)
a'
format long
p = a*inv(a)
format short
p = a.*a
a.^3
% 串联
A = [a,a]
A = [a; a]
% 复数
sqrt(-1)
c = [3+4i, 4+3j; -i, 10j]
```

M脚本



M软件运行部分结果



Syslab运行部分结果

代码选自M软件帮助文档-示例-矩阵和数组

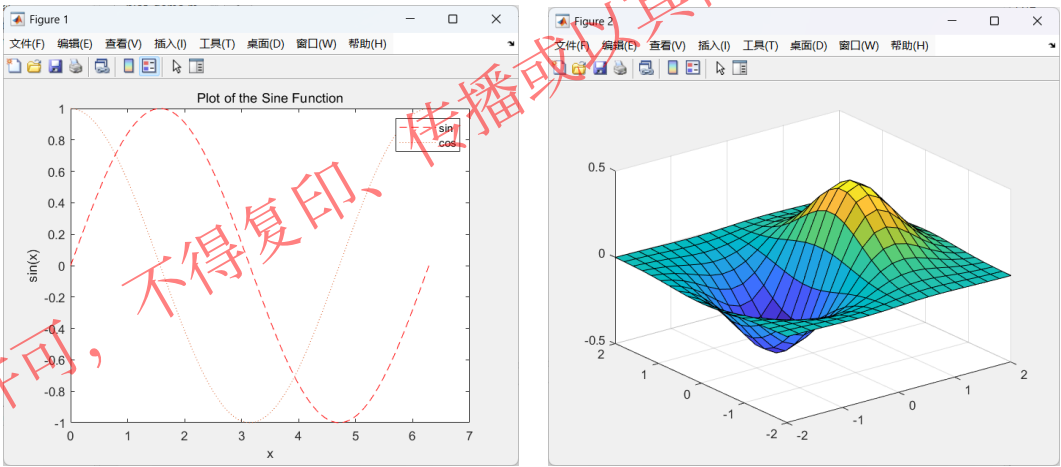


2.3 M脚本运行

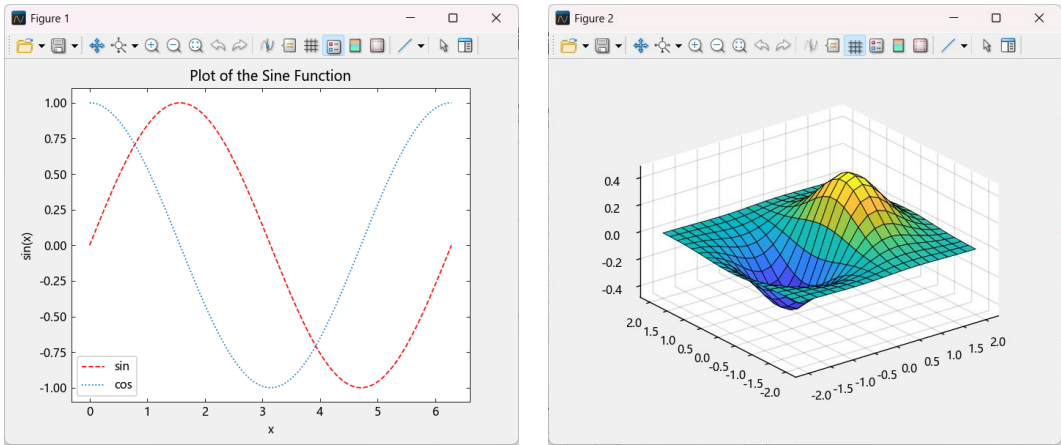
示例2：二维图和三维图绘制

```
% 线图
x = linspace(0,2*pi);
y = sin(x);
figure(1)
plot(x,y,"r--")
xlabel("x")
ylabel("sin(x)")
title("Plot of the Sine Function")
hold on
y2 = cos(x);
plot(x,y2,":")
legend("sin","cos")
% 三维绘图
x = linspace(-2,2,20);
y = x';
z = x .* exp(-x.^2 - y.^2);
figure(2)
surf(x,y,z)
```

M脚本



M软件运行结果



Syslab运行结果

代码选自M软件帮助文档-示例-二维图和三维图

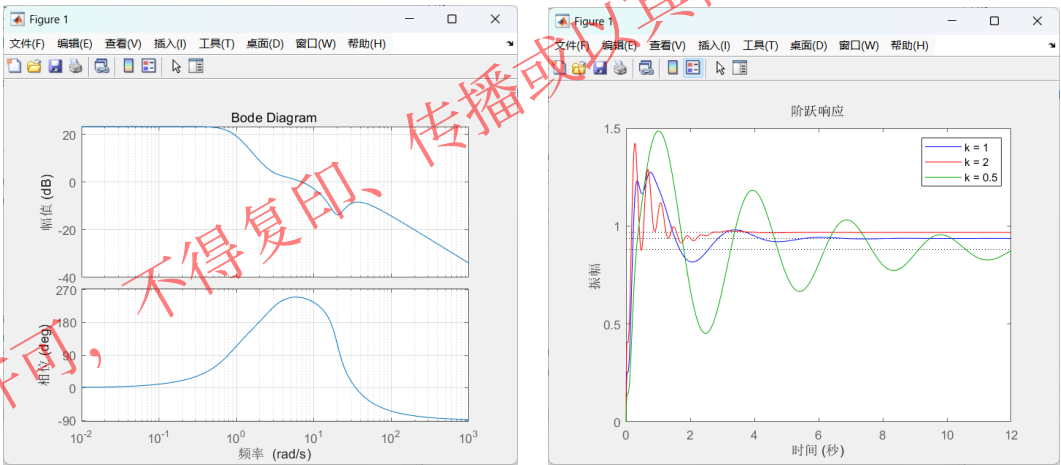


2.3 M脚本运行

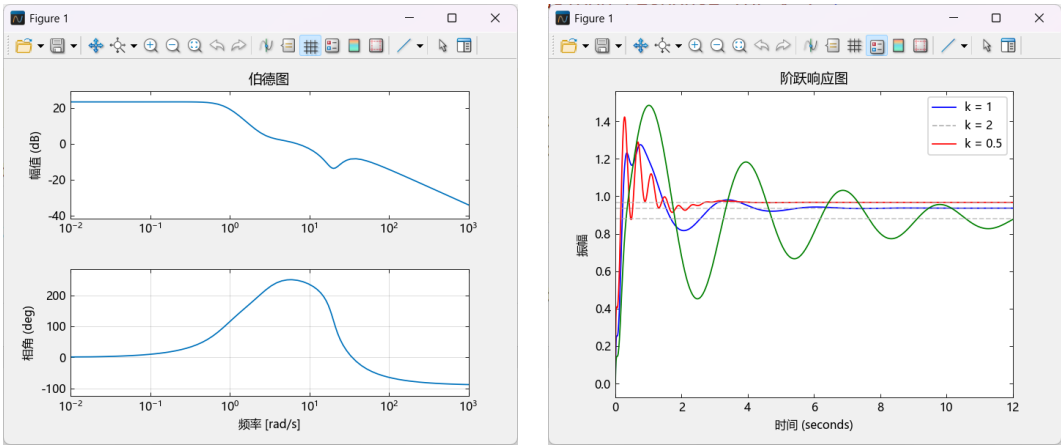
示例3：控制系统稳定性分析

```
G = tf([.5 1.3],[1 1.2 1.6 0]);
T = feedback(G,1);
pole(T)
rlocus(G)
bode(G), grid
step(T), title('Closed-loop response for k=1')
step(feedback(2*G,1)), title('Closed-loop response for k=2')
G = tf(20,[1 7]) * tf([1 3.2 7.2],[1 -1.2 0.8]) * tf([1 -8 400],[1 33 700]);
T = feedback(G,1);
step(T), title('Closed-loop response for k=1')
bode(G), grid
k1 = 2; T1 = feedback(G*k1,1);
k2 = 1/2; T2 = feedback(G*k2,1);
step(T,'b',T1,'r',T2,'g',12),
legend('k = 1','k = 2','k = 0.5')
```

M脚本



M软件运行部分结果



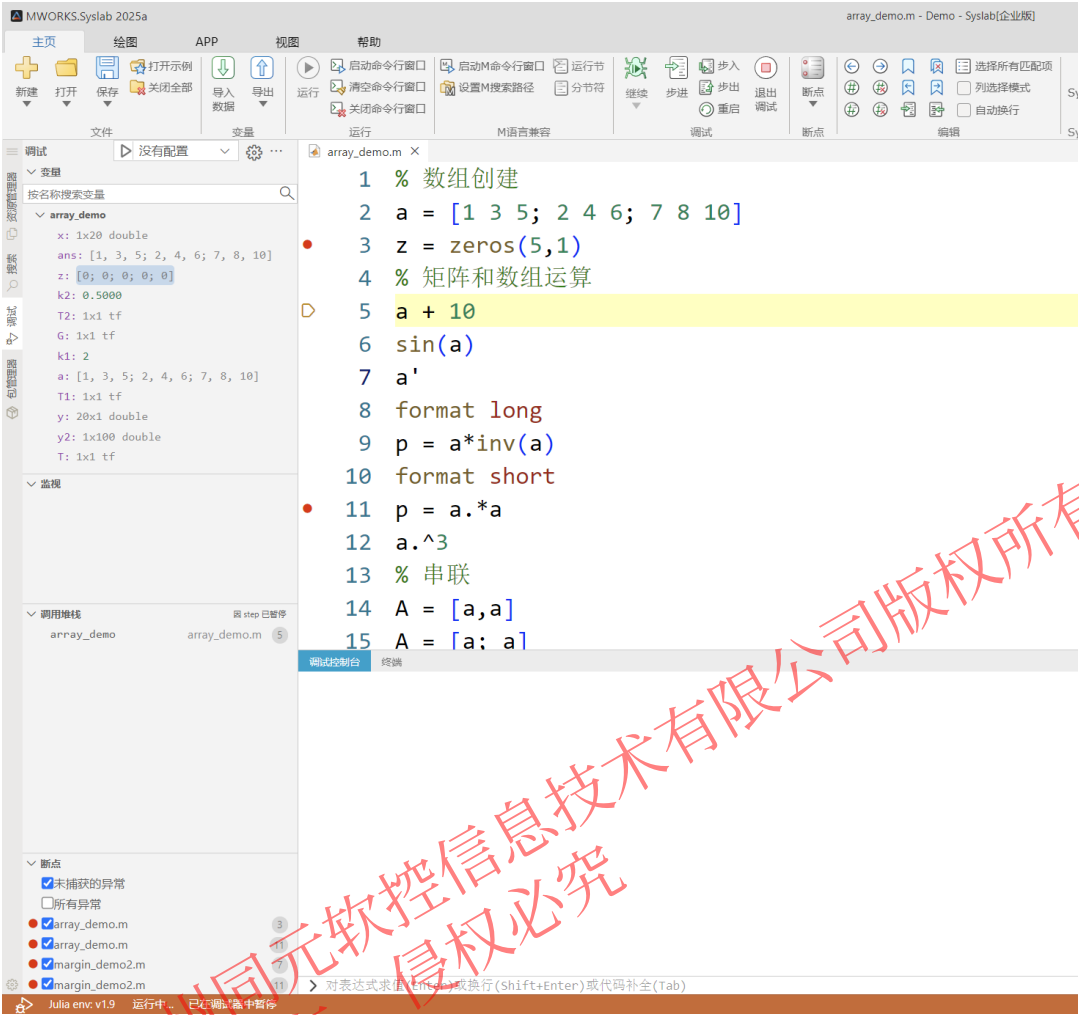
Syslab运行部分结果

代码选自M软件帮助文档-控制系统-线性分析-稳定性分析-Assessing Gain and Phase Margins



2.4 M脚本调试

支持以调试模式运行 M 代码文件，提供**单步调试**、**断点调试**、**查看调用堆栈**、**变量查询操作**等，并提供**交互式调试控制台**



操作步骤:

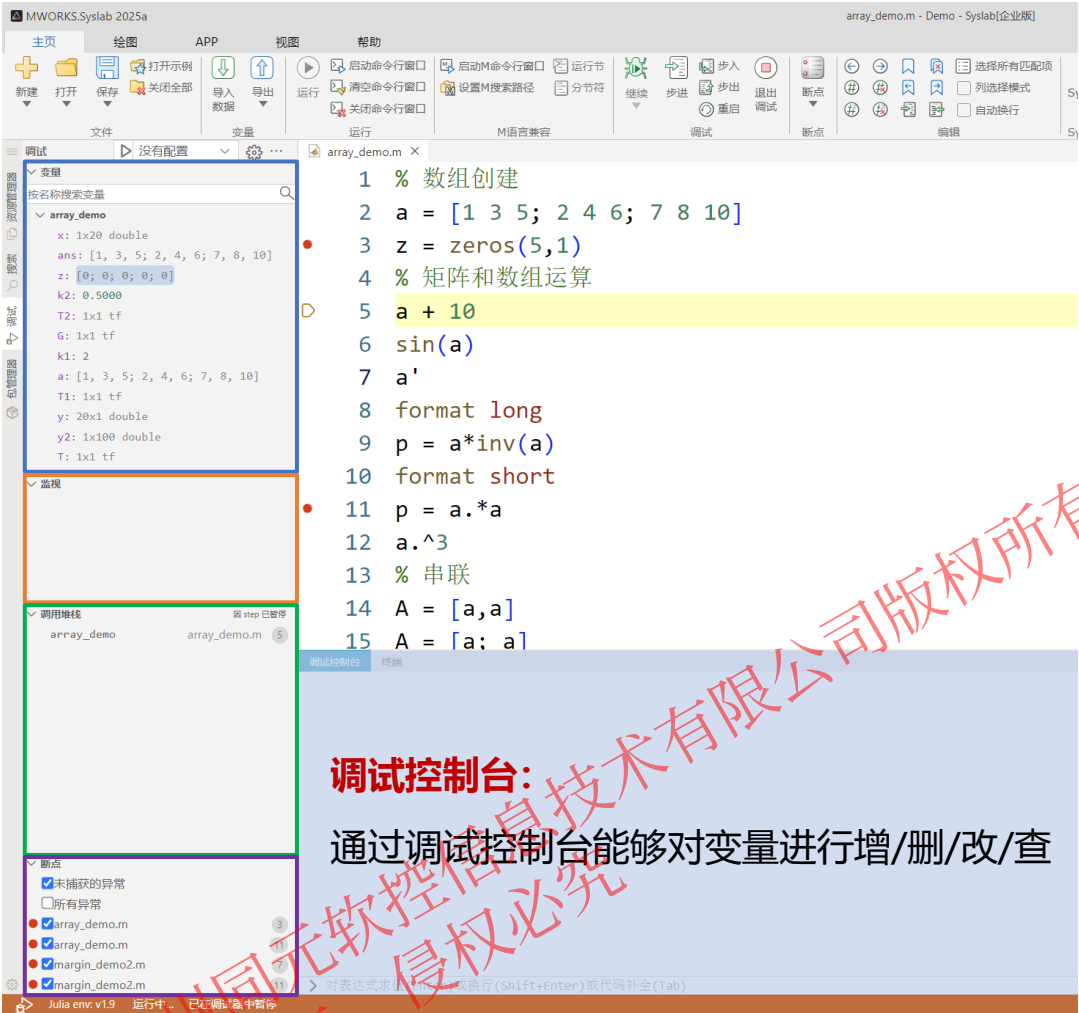
1. 在M脚本打上断点
2. 点击菜单栏中"启动调试"或F5进入DeBug模式

说明:

- **继续(F5)**: 继续运行调试
- **步进(F10)**: 单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止
- **步入(F11)**: 单步执行遇到子函数就进入并且继续单步执行
- **步出(Shift+F11)**: 当单步执行到子函数内时，执行完子函数余下部分，并返回到上一层函数
- **重启(Ctrl+Shift+F5)**: 重新启动调试
- **退出调试(Shift+F5)**: 停止调试

2.4 M脚本调试

支持以调试模式运行 M 代码文件，提供**单步调试**、**断点调试**、**查看调用堆栈**、**变量查询操作**等，并提供**交互式调试控制台**



调试控制台：
通过调试控制台能够对变量进行增/删/改/查

说明：

- **变量：**查看调试过程中的变量值
- **调用堆栈：**用于了解函数间的调用关系和逻辑
- **断点：**模型中所有断点的信息

提示：

- 变量区暂不支持显示 global 变量
- 调试控制台定义 global 变量可以采用以下写法：
global myVar; myVar = 1
- 调试控制台目前支持显示基本类型的数据，其他只显示类型和 size
- 暂不支持变量监视

目录

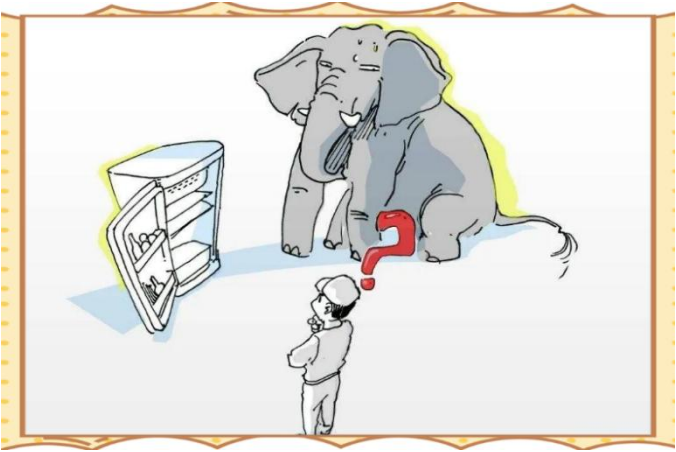
1. 基础设置

2. M脚本运行与调试

3. 高阶功能

3.1 M 语言调用 Julia 函数-将 Julia 函数封装为 M 函数

“把大象装进冰箱拢共分几步？”



“碰到 MLang 未定义和不识别的函数处理拢共分几步？”

```
demo

tymlang.VarNotFound: 变量或函数 latcfillt 无法识别
  at C:\Users\TR\Desktop\露比精选demo\演示示例\demo.m:11:7, demo
  at :1:91, demo
  at :1:91,
```



三步：



步骤	要点
1. M 类型转为 Julia 类型	M 类型如何转为 Julia 类型？有什么规则？
2. 调用 Julia 函数	如何调用已有的 Julia 库函数？ Julia 函数返回值为 Julia 类型
3. Julia 类型转为 M 类型	Julia 类型如何转为 M 类型？有什么规则？

3.1 M 语言调用 Julia 函数-第一步: M 类型转为 Julia 类型 (Julia)

Julia 函数用法

% 将 M 类型转为 Julia 类型

```
jv = Julia(o)
```

```
jv = Julia(o, jtype)
```

jv = Julia(o): 默认转换, o 为 M 类型对象, 将 o 转为 Julia 类型对象

%jv = Julia(o): 默认转换, 将转为对应类型的 Array

% Matlab中数值类型默认为 double 类型

```
jv1_1 = Julia(rand(3,1))
```

```
jcall('typeof', jv1_1)
```

```
jv1_2 = Julia(rand(1, 3))
```

```
jcall('typeof', jv1_2)
```

```
jv1_3 = Julia(1)
```

```
jcall('typeof', jv1_3)
```

jv = Julia(o, jtype): jtype 为字符串表示的 Julia 类型, 将 o 转为 类型为 jtype 的 Julia 类型对象

% Julia 的类型比较严格, 对于 Julia 函数来说, 输入参数可能是以下三类: 标量、向量、数组 (不含向量)

% M 的向量类型, 无论行列向量都可以转为 Julia 的向量 (Vector)

```
jv2_1 = Julia(rand(3,1), 'Vector{Float64}')
```

```
jcall('typeof', jv2_1)
```

```
jv2_2 = Julia(rand(1, 3), 'Vector')
```

```
jcall('typeof', jv2_2)
```

```
jv2_3 = Julia(1, 'Int64')
```

```
jcall('typeof', jv2_3)
```

% Mlang 与 Julia 数据类型对应关系

double -> Float64

single -> Float32

complex -> ComplexF64

string -> String

char -> String

uint8 -> UInt8

uint16 -> UInt16

uint32 -> UInt32

uint64 -> UInt64

int8 -> Int8

int16 -> Int16

int32 -> Int32

int64 -> Int64

struct -> NamedTuple

红色为暂不支持转换类型

3.1 M 语言调用 Julia 函数-第二步：调用 Julia 函数 (jcall)

jcall 函数用法

```
% 调用 Julia 函数
jcall(expr, jv1, ..., jvn)
jcall(__, '-kwargs', kws)
jcall('-return', mtype, __)
jcall('-vector', __)
jcall('-noreturn', __)
```

```
jcall(expr, jv1, ..., jvn)

% 创建 类型为 Vector{Int} 的 Julia 对象
A = Julia([2,1,3], 'Vector{Int}');
% 调用 Julia 函数需输入 Julia 对象
jv1 = jcall('TyBase.sort', A)
```

```
% M类型默认转换为 Julia 类型，转换后为矩阵
A = Julia([2,1,3]);
jv1 = jcall('TyBase.sort', A)
```

```
TyMLang.JuliaError: UndefKeywordError: UndefKeywordError: keyword argument `dims` not assigned

at /jcall.m:2:18, v2_call
at /jcall.m:2:18, jcall
at C:\Users\TR\Desktop\Untitled2.m:9:5, jcall
at C:\Users\TR\Desktop\Untitled2.m:9:5, Untitled2
```

```
jcall(__, '-kwargs', kws)

A = Julia([2,1,3]);
jv1 = jcall('TyBase.sort', A)
% 关键字参数为结构体，字段名为 Julia 函数中关键字的 key 值
jkwargs.dims = Julia(2, 'Int');
% '-kwargs' 标识关键字参数
value2 = jcall('TyBase.sort', A, '-kwargs', jkwargs)
```



```
% 运行结果
jv1 =
1x1 domain_proxy
 [1, 2, 3]
```



3.1 M 语言调用 Julia 函数-第三步: Julia 类型转为 M 类型 (fromJulia)

fromJulia 函数用法

% 将 Julia 类型转为 M 类型

```
o = fromJulia(jv)
```

```
o = fromJulia(jv, mtype)
```

`o = fromJulia(jv)`: 默认转换, `jv` 为 Julia 类型对象, 将 `jv` 转为 M 类型对象

`o = fromJulia(jv, mtype)`: 将 `jv` 转为类型为 `mtype` 的 M 类型对象, 此场景较为少用

% M 类型不需要区分标量、向量和数组, 可以直接转为 M 的对应类型的数组

```
jv1_1 = Julia(rand(3,1))
```

```
jcall('typeof', jv1_1)
```

```
o1_1 = fromJulia(jv1_1)
```

```
class(o1_1)
```

```
jv1_2 = Julia(1)
```

```
jcall('typeof', jv1_2)
```

```
o1_2 = fromJulia(jv1_2)
```

```
class(o1_2)
```

% 使用场景较少, 一般自动转换就足够使用

% 该用法用于有时 Julia 返回值为不确定, 可能为 Int、Float64、ComplexF64, 但是 M 代码中希望值为 double 类型

% Julia 的 Vector 转为 M 类型时转为行向量:

```
jv2_1 = Julia([1,2,3])
```

```
jcall('typeof', jv2_1)
```

```
o2_1 = fromJulia(jv2_1, 'double')
```

```
class(o2_1)
```

% Julia 与 MLang 数据类型对应关系

Float64 -> double

Float32 -> single

ComplexF64 -> complex

String -> string

% Char

UInt8 -> uint8

UInt16 -> uint16

UInt32 -> uint32

UInt64 -> uint64

Int8 -> int8

Int16 -> int16

Int32 -> int32

Int64 -> int64

% Struct(NamedTuple)

红色为暂不支持转换类型

3.1 M 语言调用 Julia 函数-示例: latcfilt 格型滤波器

latcfilt

晶格和晶格梯形滤波器实现

语法

```
[f,g] = latcfilt(k,x)
[f,g] = latcfilt(k,v,x)
[f,g] = latcfilt(k,1,x)
[f,g,zf] = latcfilt(__,"ic",zi)
[f,g,zf] = latcfilt(__,dim)
```

描述

[f, g] = latcfilt (k, x) 使用指定的 FIR 晶格系数对输入信号进行滤波, 并返回前向晶格滤波器结果和后向滤波结果。

例子

▼ FIR 晶格滤波器

生成具有 512 个高斯白噪声样本的信号。

```
x = randn(512,1);
```

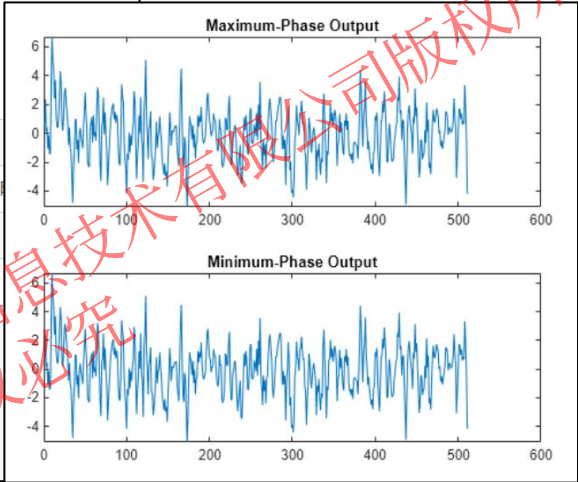
使用 FIR 晶格滤波器过滤数据。指定反射系数, 使晶格滤波器等效于三

```
[f,g] = latcfilt([1/2 1],x);
```

在单独的绘图中绘制晶格滤波器的最大相位和最小相位输出

```
subplot(2,1,1)
plot(f)
title('Maximum-Phase Output')

subplot(2,1,2)
plot(g)
title('Minimum-Phase Output')
```



```
x = randn(512,1);
[f,g] = latcfilt([1/2 1],x);
subplot(2,1,1)
plot(f)
title('Maximum-Phase Output')
subplot(2,1,2)
plot(g)
title('Minimum-Phase Output')
```

tymlang.VarNotFound: 变量或函数 latcfilt 无法识别
at C:\Users\TR\Desktop\露比精选demo\1. 类型转换\Unnamed.m:64:7, Unnamed

```
k=[1/2 1]
% 输入参数转为 Julia 类
jv_k = Julia(k);
jv_x = Julia(x)
% 根据 Syslab 帮助文档, 该函数有多个输出
jv = jcall('TySignalProcessing.latcfilt', jv_k, jv_x);
% 使用 jindex 将多个输出拆开
jv_1= jindex(jv, 1);
jv_2= jindex(jv, 2);
% 将输出参数转为 M 类型
f = fromJulia(jv_1);
g = fromJulia(jv_2);
```


3.1 M 语言调用 Julia 函数-未兼容原因及处理

未兼容原因	报错详情	示例	解决方案
函数未支持	tymlang.VarNotFound: 变量或函数**无法识别/未定义	<pre>x = randn(512,1); [f,g] = latcfilt([1/2 1],x); subplot(2,1,1) plot(f) title('Maximum-Phase Output') subplot(2,1,2) plot(g) title('Minimum-Phase Output')</pre>	1. 将 Julia 函数封装为 M 函数 2. 联系同元工程师
参数异常	TyMLang.ArgParserError: 期望参数是**类型/输入参数不足	<pre>[X,Y,Z] = peaks(25); CO(:, :, 1) = zeros(25); CO(:, :, 2) = ones(25).*linspace(0.5,0.6,25); CO(:, :, 3) = ones(25).*linspace(0,1,25); s=surf(X,Y,Z,'CData',CO)</pre>	函数未封装完全，原因是Julia侧函数还未支持此类用法，请联系同元工程师
JuliaError	TyMLang.JuliaError:Exception: 创建或更新**时出错，数组的形状或大小错误	<pre>[X,Y,Z] = peaks(25); load moonalb20c.mat; surf(X,Y,Z,moonalb20c);</pre>	Julia侧的报错，原因是Julia函数有这种用法，但是使用有误，这种用法在M里面一般也会报错
未找到xx	tymlang.PropertyNotFound: 属性**未在 GraphicObject 中找到	<pre>% ax Ax = gca; ax.XTick=[];</pre>	目前不支持这种 . 索引 替代方案： set(ax, 'XTick', [])



3.2 Julia语言调用M函数

支持在 Julia 中执行 M 代码、操作 M 语言的对象、类型转换以及调用 M 语言的函数

函数	语法	说明
eval_mcode!	<code>eval_mcode!(code)</code>	在 Julia 中直接运行 M 代码，返回值为 <code>nothing</code>
to_mlang	<code>to_mlang(x)</code>	将一个 Julia 对象转换为 <code>MxArray</code> 类型，支持数值类型和对应的数组，以及标量的 <code>String</code> 类型
from_mlang	<code>from_mlang(pm::MxArray)</code> <code>from_mlang(T::Type, pm::MxArray)</code>	- 将 <code>MxArray</code> 类型转换对应的 Julia 类型，支持数值类型和对应的数组类型，以及标量的字符串，其他类型仍返回 <code>MxArray</code> - 将 <code>MxArray</code> 类型转换为 <code>T</code> 类型，支持数值类型和对应的数组类型，以及标量的 <code>String</code> 类型，类型不匹配时返回 <code>nothing</code>
mexCallMLang	<code>mexCallMLang(funName::String, nargsout::Integer, args::Vector{MxArray})</code>	<code>funName</code> 为调用的 M 函数， <code>nargsout</code> 为 M 函数的返回参数个数， <code>args</code> 为调用的 M 函数的输入参数，返回值为 <code>Vector{MxArray}</code>
mexGetVar	<code>mexGetVar(varName::String)</code>	获取 <code>MLang REPL</code> 中的变量，返回值为 <code>MxArray</code>
mxget_cell	<code>mxget_cell(pm::MxArray, i)</code>	获取元胞数组 <code>pm</code> 的第 <code>i</code> 个元素，返回值为 <code>MxArray</code> 类型
mxset_cell	<code>mxset_cell(pm::MxArray, i, value::MxArray)</code>	将元胞数组 <code>pm</code> 的第 <code>i</code> 个元素设置为 <code>value</code>
mxget_field	<code>mxget_field(pm::MxArray, i, fieldname)</code>	获取结构体 <code>pm</code> 的第 <code>i</code> 个元素的 <code>fieldname</code> 字段，返回值为 <code>MxArray</code> 类型
mxset_field	<code>mxset_field(pm::MxArray, i, fieldname, value::MxArray)</code>	将结构体 <code>pm</code> 的第 <code>i</code> 个元素的 <code>fieldname</code> 字段设置为 <code>value</code>
其他 API	详见帮助文档	

兼容局限性

分类	详情	示例
Win7 操作系统的局限性	- 日期和文件系统API兼容的局限性 - 在 Win7 环境下，解析器不是并行的，解析较多文件耗时更长 - 在 Win7 环境下，暂不支持 tar 命令	- 对于dir返回的文件/文件夹项，date一律为 1900-01-01，datenum一律为 0 - now函数等时间相关的函数无法使用
数据类型的局限性	- M 语言计算环境暂不支持部分 MATLAB 数据类型，如：table, datetime, categorical 等 - M 语言计算环境不支持导入由部分经 MATLAB 保存至 MAT 文件的数据类型，如：ss, tf, zpk 等	- datetime 是支持单独使用，但是不支持传入到函数里面 - table目前还未支持
功能的局限性	- M 语言计算环境暂不支持部分 MATLAB 功能，如：并行计算、APP 构建等 - M 语言计算环境工作区、调试变量面板暂不支持显示 global 变量，在命令行和调试控制台定义 global 变量可以采用以下写法：global myVar; myVar = 1 - M 语言计算环境在脚本文件中使用 input 函数时，第一个 input 需要输入两次	input函数的输入属于Julia的调用问题

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听